
```

clear, clc, close all

driveLetter = 'F';

parentFile = {[driveLetter
':\SPACL\impacting_droplet_experiment\heating80C_droplet50mm\IPDI\heat008'
[driveLetter
':\SPACL\impacting_droplet_experiment\heating80C_droplet50mm\IPDI\heat016'
[driveLetter
':\SPACL\impacting_droplet_experiment\heating80C_droplet50mm\IPDI\heat024'
[driveLetter
':\SPACL\impacting_droplet_experiment\heating80C_droplet50mm\DDI\heat004'
[driveLetter
':\SPACL\impacting_droplet_experiment\heating80C_droplet50mm\DDI\heat008'
[driveLetter
':\SPACL\impacting_droplet_experiment\heating80C_droplet50mm\DDI\heat012'
[driveLetter
':\SPACL\impacting_droplet_experiment\heating80C_droplet50mm\DDI\heat016']]];

% parentFile = {[driveLetter
':\SPACL\impacting_droplet_experiment\heating80C_droplet50mm\IPDI\heat008']]];

ATV = 0.185;

FPS = 15000;

for i = 1:length(parentFile)
    clc, disp(parentFile{i})
    %     funcGetCalibration(parentFile{i})
    %     funcGetContactLines(parentFile{i})
    %     funcCropImg(parentFile{i})
    %     funcDropletProcessing(parentFile{i}, FPS, ATV)
    funcPowerLawSpread(parentFile{i})
end

%
load(fullfile(parentFile{1}, 'heat024_contactLine.mat'), 'yline', 'xline', 'a', 'b'
)
% num = 1;
% yline = yline{num};
% xline = xline{num};
% a = a(num);
% b = b(num);
%
% IO = flipud(imread("Spreading000100.tif"));
% im = Spread_PreProcessing(IO, ATV);
% figure;
% imshow(im)
% hold on
% plot(xline, yline)
% hold off

```

```
% figure;  
% imshow(I0)
```

Published with MATLAB® R2026a

```

function funcGetCalibration(parentFile)

    files = dir(parentFile);
    files(1:2) = [];
    subFilesStruct = files(~[files.isdir]);
    fileArr = contains(convertCharsToStrings({subFilesStruct.name}), "calib")
    & endsWith(convertCharsToStrings({subFilesStruct.name}), ".tif");
    imgFileDir =
    fullfile(subFilesStruct(fileArr).folder, subFilesStruct(fileArr).name);

    dataFile1 = imread(imgFileDir);
    dataFile2 = imadjust(dataFile1);
    dataFile3 = dataFile2';
    dataFile3 = dataFile3(:,150:end);

    for i = 1:4
        figure;
        imshow(dataFile3), axis on
        set(gca, 'YDir', 'normal') %Set Axis to normal so image is displayed
correctly
        drawnow
        hold on
        uiwait(helpdlg('Zoom in before closing this box'))

        [xc(i),yc(i)] = ginput(1);
        close all
    end

    xDist = size(dataFile2,2);
    xline = linspace(1,xDist,5000);
    coefficients1 = polyfit([xc(1), xc(2)], [yc(1), yc(2)], 1); %Linear Fit
of Points
    coefficients2 = polyfit([xc(3), xc(4)], [yc(3), yc(4)], 1); %Linear Fit
of Points
    a1 = coefficients1(1); b1 = coefficients1(2);
    y1=a1*xline+b1;
    a2 = coefficients2(1); b2 = coefficients2(2);
    y2=a2*xline+b2;

    diffy = mean(abs(y2-y1));
    prompt = 'HOW MANY LINE PAIRS?';
    numLines = input(prompt);
    calib = diffy/numLines;

    writematrix(calib,[imgFileDir(1:end-4), '.txt'])

end

```

Published with MATLAB® R2026a

```

function funcGetContactLines(parentFile)

    files = dir(parentFile);
    files(1:2) = [];
    subFoldersStruct = files([files.isdir]);
    subFolders =
fullfile(convertCharsToStrings({subFoldersStruct.folder}),convertCharsToString
s({subFoldersStruct.name}),'Spreading');
    xline = cell(length(subFolders),1); yline = cell(length(subFolders),1);
    a = NaN(length(subFolders),1); b = NaN(length(subFolders),1);

    for i = 1:length(subFolders)

        firstImg = dir(subFolders(i));
        firstImg = fullfile(firstImg(end).folder,firstImg(end).name);
        I0=imread(firstImg);
        Iflip = im2uint8(mat2gray(I0));
        Filled = imcomplement(imfill(imcomplement(Iflip),'holes')); %Fill
Holes in Objects, imfill auto assumes holes are light on black background
        I=flipud(Filled); %Flip Matrix in Original Image

        figure;
        imshow(I); axis on
        set(gca,'YDir','normal') %Set Axis to normal so image is displayed
correctly
        drawnow

```

interactive crop

```

hold on
uiwait(helpdlg('Zoom in before closing this box'))

for j=1:2
    [x(j),y(j)] = ginput(1);
    plot(x,y,'o','color','red')
end

widthImage = size(I,2);
xlinef=linspace(1,widthImage,widthImage); %Linear Spaced Vector
coefficients = polyfit([x(1), x(2)], [y(1), y(2)], 1); %Linear Fit of
Points
af = coefficients(1);
bf = coefficients(2);
ylinef=round(af*xlinef+bf);
plot(xlinef,ylinef,'magenta','LineWidth',2);

uiwait(helpdlg('Happy with contact line?'))
hold off
close(gcf)

yline{i} = ylinef;
xline{i} = xlinef;

```

```
    a(i) = af;  
    b(i) = bf;  
  
end  
  
[~,testName] = fileparts(parentFile);  
testName = [testName '_firstLine.mat'];  
save(fullfile(parentFile,testName),'yline','xline','a','b')  
  
end
```

Published with MATLAB® R2026a

```
function funcCropImg(parentFile)

    Master = parentFile;
    files = dir(Master);
    files(1:2) = [];
    subFilesStruct = files(~[files.isdir]);
    subFoldersStruct = files([files.isdir]);
    subFolders =
fullfile(convertCharsToStrings({subFoldersStruct.folder}),convertCharsToStrings({subFoldersStruct.name}));

    a = NaN(length(subFolders),1); b = NaN(length(subFolders),1);
    fileArr1 =
contains(convertCharsToStrings({subFilesStruct.name}),'firstLine');
    contactLineFileDir =
fullfile(subFilesStruct(fileArr1).folder,subFilesStruct(fileArr1).name);
    load(contactLineFileDir,'yline','xline')

    for k=1:length(subFolders)
        fprintf('Droplet %1.0f\n',k)
        s1=subFolders(k);
        cropFreefall(s1,yline{k})
        [xline{k},yline{k},a(k),b(k)] = cropSpreading(s1,xline{k}, yline{k});
    end

    newContactLineFileDir =
replace(contactLineFileDir,'firstLine','contactLine');
    save(newContactLineFileDir,'xline','yline','a','b')
end
```

Published with MATLAB® R2026a

```
function cropFreefall(s1, yline)
```

get a valid folder name

```
    if ~exist('images_folder','var')
        [N,images_folder,image_files]=openFolder(s1,'Freefall');
    end
    if ~exist(images_folder,'dir')
        error(['The images folder name ' images_folder ' is not a valid
folder'])
    end
```

then, get the name of the folder to save to

```
    saveFolder = fullfile(fileparts(images_folder),'Freefall Cropped');
    mkdir(saveFolder);
```

Next, for every image file, perform crop, and save the cropped image.

```
    for i=1:N
        % read a single file
        image_file_1_full_name=fullfile(images_folder,image_files(i).name);
        I0=imread(image_file_1_full_name);
        Iflip = im2uint8( mat2gray(I0));
        Filled = imcomplement(imfill(imcomplement(IFlip),'holes')); %Fill
Holes in Objects, imfill auto assumes holes are light on black background
        I=flipud(Filled); %Flip Matrix in Original Image

        % Turn every pixel below the contact line white
        I2=I;
        I2(1:(min(yline)-1),:) = [];
        I3=flipud(I2);

        % and write it to file

        image_file_1_full_name_to_save2=fullfile(saveFolder,image_files(i).name);
        imwrite(I3,image_file_1_full_name_to_save2)
        if i==N, pause(0.5), end
    end
end
```

Published with MATLAB® R2026a

```
function [xline,yline,a,b] = cropSpreading(s1,xline,yline)
```

get a valid folder name

```
    if ~exist('images_folder','var')
        [N,images_folder,image_files]=openFolder(s1,'Spreading');
    end
    if ~exist(images_folder,'dir')
        error(['The images folder name ' images_folder ' is not a valid
folder'])
    end
```

then, get the name of the folder to save to

```
    saveFolder = fullfile(fileparts(images_folder),'Spreading Cropped');
    mkdir(saveFolder);
```

Next, for every image file, perform crop, and save the cropped image.

```
    imgFile =
fullfile(convertCharsToStrings({image_files.folder}),convertCharsToStrings({image_files.name}));

    I1 = flipud(im2uint8(mat2gray(imread(imgFile(1)))));
    I1_filled = imcomplement(imfill(imcomplement(I1),'holes'));
    I2 = imbinarize(I1_filled,0.25); %Convert Image to black and white

    imgHeight = size(I2,1);

    currRow = NaN(imgHeight,1);

    for i = 1:imgHeight
        currRow(i) = sum(I2(i,:)==0)>0;
    end

    indexTop = find(currRow,1,'last');

    J1 = flipud(im2uint8(mat2gray(imread(imgFile(end)))));
    J1_filled = imcomplement(imfill(imcomplement(J1),'holes'));
    J2 = imbinarize(J1_filled,0.25); %Convert Image to black and white

    imgWidth = size(J2,2);

    for i = 1:imgWidth
        currCol(i) = sum(J2(:,i)==0)>0;
    end

    indexLeft = find(currCol,1,'first');
```

```

indexRight = find(currCol,1,'last');

effectRad = (indexRight-indexLeft+1)/2;
newLeft = floor(indexLeft-0.05*effectRad);
newRight = ceil(indexRight+0.05*effectRad);
if newLeft<1,newLeft=1;end
if newRight>imgWidth,newRight=imgWidth;end

yAdjust = min(yline)-1;
xline = xline(:,newLeft-(newLeft-1):newRight-(newLeft-1));
yline = yline(:,newLeft:newRight)-yAdjust;
coefficients = polyfit([xline(1), xline(end)], [yline(1), yline(end)],
1); %Linear Fit of Points
a = coefficients(1);
b = coefficients(2);

for i = 1:N

    K1 = im2uint8(mat2gray(imread(imgFile(i)))));
    K2 = flipud(K1);
    K3 = K2(yAdjust+1:indexTop+5,newLeft:newRight);
    K4 = flipud(K3);

    imwrite(K4,fullfile(saveFolder,image_files(i).name))
end

end

```

Published with MATLAB® R2026a

```
function funcDropletProcessing(parentFile,fps,ATV)
```

Initialize Variables

```
FPS = fps;

% load from matlab variable
files = dir(parentFile);
files(1:2) = [];
subFilesStruct = files(~[files.isdir]);
subFoldersStruct = files([files.isdir]);
subFolders =
fullfile(convertCharsToStrings({subFoldersStruct.folder}),convertCharsToStrings({subFoldersStruct.name}));

% load contact line
fileArr1 =
contains(convertCharsToStrings({subFilesStruct.name}),"contactLine");
contactLineFileDir =
fullfile(subFilesStruct(fileArr1).folder,subFilesStruct(fileArr1).name);
load(contactLineFileDir,'yline','xline','a','b')

% load calibration value
fileArr2 = contains(convertCharsToStrings({subFilesStruct.name}),"calib")
& endsWith(convertCharsToStrings({subFilesStruct.name}),".txt");
calibrationFileDir =
fullfile(subFilesStruct(fileArr2).folder,subFilesStruct(fileArr2).name);
Calibration = load(calibrationFileDir);

clearvars -except Calibration FPS Master subFolders yline xline a b ATV
parentFile
```

Main Code

```
Test=string(length(subFolders));
ThresholdingValue=zeros(length(subFolders),1);
FreefallDiameter=zeros(length(subFolders),1);
VelocityImpact=zeros(length(subFolders),1);
timeStarPastLineArr=zeros(length(subFolders),1);

TimeStarData = cell(length(subFolders),1);
SpreadData = cell(length(subFolders),1);
EdgeVelocityData = cell(length(subFolders),1);
thetaLeftData = cell(length(subFolders),1);
thetaRightData = cell(length(subFolders),1);

% Freefall Analysis
for k=1:length(subFolders)
    clc
    s1=convertStringsToChars(subFolders(k));
    fprintf('Processing Freefall: %s\n',s1)
```

```

        [D_Falling, ImpactVelocity, timeStarPastLine] =
FreefallModified(Calibration, FPS, s1);
        FreefallDiameter(k)=D_Falling;
        VelocityImpact(k)=ImpactVelocity;
        timeStarPastLineArr(k) = timeStarPastLine;
    end

    for k=1:length(subFolders)
        clc, close all
        s1=convertStringsToChars(subFolders(k));
        %spreading analysis
        fprintf('Processing Spreading: %s\n', s1)
        [TimeStar, Spread, EdgeVelocity, thetaLeft, thetaRight] =
DynamicSpreading(s1, ATV, FPS, Calibration, xline{k}, yline{k}, a(k), b(k), FreefallDi
ameter(k), VelocityImpact(k), timeStarPastLineArr(k));
        [~, sampleName] = fileparts(s1);

        %Excel Sheet Variables
        Test(k)=string(sampleName); %Test File name
        ThresholdingValue(k)=ATV;

        TimeStarData{k} = TimeStar;
        SpreadData{k} = Spread;
        EdgeVelocityData{k} = EdgeVelocity;
        thetaLeftData{k} = thetaLeft;
        thetaRightData{k} = thetaRight;

        TimeStarCell=num2cell(TimeStar);
        SpreadCell=num2cell(Spread);
        EdgeVelocityCell=num2cell(EdgeVelocity);
        thetaLeftDegCell=num2cell(thetaLeft);
        thetaRightDegCell=num2cell(thetaRight);

        %Write To Sheet2
        A=[sampleName, 't*', TimeStarCell];
        B=[sampleName, 'Spread', SpreadCell];
        C=[sampleName, 'Edge Velocity', EdgeVelocityCell];
        D=[sampleName, 'Left DCA', thetaLeftDegCell];
        E=[sampleName, 'Right DCA', thetaRightDegCell];
        F=vertcat(A, B, C, D, E);

        txt1=[getExcelCol(1+6*(k-1)), '1'];
        filename = fullfile(parentFile, 'dataSheet.xlsx');
        writetable(F, filename, 'Sheet', 2, 'Range', txt1);
    end
end

```

Concatenate Arrays

```

A = [Test.' FreefallDiameter VelocityImpact ThresholdingValue];
T = array2table(A, 'VariableNames', {'Test',
'DiameterFalling', 'VelocityImpact', 'ThresholdingValue'});
filename = fullfile(parentFile, 'dataSheet.xlsx');
writetable(T, filename, 'Sheet', 1, 'Range', 'B2')

```

```
save(fullfile(parentFile, 'dataMat.mat'), 'TimeStarData', 'SpreadData', 'EdgeVelocityData', 'thetaLeftData', 'thetaRightData')
```

```
end
```

Published with MATLAB® R2026a

```
function [Diameter, ImpactVelocity,timeStarPastLine]=  
Freefall(Calibration,FPS,s1)
```

```
    Rmin = 25;  
    Rmax = 400;
```

```
    [N,images_folder,image_files]=openFolder(s1,'Freefall Cropped');
```

Calculate Droplet Dimensions and Location for Every Picture.

```
    topDrop=[];  
    bottomDrop=[];  
    radArr = NaN(1,length(N));  
  
    for i=1:N  
        % read a single file  
        image_file_1_full_name=fullfile(images_folder,image_files(i).name);  
        I0=imread(image_file_1_full_name);  
  
        I = im2uint8( mat2gray(I0)); %Convert Image to 8bit  
        BW =  
imbinarize(I,'adaptive','ForegroundPolarity','dark','Sensitivity',0.4);  
        BW2 = imcomplement(bwareaopen(imcomplement(BW),1000,4)); %Remove  
Objects below 2000 pixels, bwareaopen auto assumes objects are white  
        se = strel('disk',10);  
        closeBW = imcomplement(imclose(imcomplement(BW2),se));  
        BW3 = imcomplement(imfill(imcomplement(closeBW),'holes')); %Fill  
Holes in Objects, imfill auto assumes holes are light on black background  
        im=BW3;  
  
        %Finding Radius and Center Coordinates  
        [~,radius] = imfindcircles(im,[Rmin  
Rmax], 'ObjectPolarity','dark','Sensitivity',0.92);  
  
        %Creating Arrays of Image Iteration and Radii  
        radArr(i)=radius; %Creating Radii Location Array  
  
        if i==1  
            droplet = sum(imcomplement(im')) > 0;  
            topDrop(1) = find(droplet, 1, 'first');  
            topDrop(2) = find(droplet, 1, 'last');  
        end  
  
        if i==N  
            droplet = sum(imcomplement(im')) > 0;  
            bottomDrop(1) = find(droplet, 1, 'first');  
            bottomDrop(2) = find(droplet, 1, 'last');  
  
            distPixRemain = length(droplet)-bottomDrop(2);  
        end  
    end
```

```
end

%Converting Edge Locations from Pixels to mm
Diameter=mean(radArr./Calibration*2); %%milimeters
ImpactVelocity=mean(bottomDrop-topDrop)/Calibration/1000*FPS/(N-1);
%meters per second

%Get Tstar leftover
distRemain = distPixRemain/Calibration/1000; %meters
timeRemain = distRemain/ImpactVelocity; %sec
dt = 1/FPS; %sec
timeStarPastLine = abs(rem(timeRemain,dt)-dt)*ImpactVelocity/Diameter;

end
```

Published with MATLAB® R2026a

```
function im=Spread_PreProcessing(I0,ATV)
    %Preprocessing Image
    I = im2uint8( mat2gray(I0)); %Convert Image to 8bit
    BW = imbinarize(I,ATV); %Convert Image to black and white;
    indexCol = NaN(size(BW,2),1);
    for i = 1:size(BW,2)
        col = imcomplement(BW(:,i));
        index = find(col,1, 'last');
        if isempty(index)
            continue;
        else
            indexCol(i) = index;
        end
    end

    indexCol = indexCol(~isnan(indexCol));
    modeRow = mode(indexCol);
    row = imcomplement(BW(modeRow,:));
    indexLeft = find(row,1, 'first');
    indexRight = find(row,1, 'last');
    BW(modeRow,indexLeft:indexRight) = 0;

    bottomStrip = imcomplement(BW(1,:));
    xLeft = find(bottomStrip,1, 'first');
    xRight = find(bottomStrip,1, 'last');
    BW(1,xLeft:xRight) = 0;

    BW2 = imcomplement(bwareaopen(imcomplement(BW),1000,4)); %Remove Objects
    below 4000 pixels, bwareaopen auto assumes objects are white
    se = strel('disk',50);
    closeBW = imcomplement(imclose(imcomplement(BW2),se));
    im = imcomplement(imfill(imcomplement(closeBW), 'holes'));

end
```

Published with MATLAB® R2026a

```
function [TimeStar,Spread,EdgeVelocity,thetaLeft,thetaRight] =
DynamicSpreading(s1,ATV,FPS,Calibration,xline,yline,a,b,D_o,U_o,t_pastLine)
    [N,images_folder,image_files]=openFolder(s1,'Spreading Cropped');

    D_pix = NaN(1,length(N));
    thetaLeft = NaN(1,length(N));
    thetaRight = NaN(1,length(N));

    for i=1:N
        % read a single file
        imgFile=fullfile(images_folder,image_files(i).name);
        I0flip=imread(imgFile); %Read a single image
        I0=flipud(I0flip); %Flip Matrix in Original Image
        I1 = Spread_PreProcessing(I0,ATV);

        [leftDCA,rightDCA] = DynamicContactAngle(xline,yline,I1);
        D_pix(i) = getSpread(I1,xline,a,b);
        thetaLeft(i) = leftDCA;
        thetaRight(i) = rightDCA;
    end

    [TimeStar,Spread,EdgeVelocity] =
    SpreadCalcs(D_pix,Calibration,FPS,U_o,D_o,t_pastLine);

end
```

Published with MATLAB® R2026a

```
function S = getSpread(im,xline,a,b)

    dropSurf = round(xline*a+b);
    surfStrip = NaN(length(dropSurf),1);
    for i = 1:length(dropSurf)
        surfStrip(i) = im(dropSurf(i),xline(i));
    end

    surfStrip = imcomplement(surfStrip);

    xLeftPt = find(surfStrip,1, 'first');
    xRightPt = find(surfStrip,1, 'last');
    yLeftPt = xLeftPt*a+b;
    yRightPt = xRightPt*a+b;

    S = sqrt((xRightPt-xLeftPt)^2+(yRightPt-yLeftPt)^2);
end
```

Published with MATLAB® R2026a

```

function [thetaLeftDeg,thetaRightDeg] = DynamicContactAngle(xCL,yCL,im)

%make everything below CL == white
for i = 1:length(xCL)
    if yCL(i)>1,im(1:yCL(i)-1,i)=true;end
end

imRot = flipud(imrotate(im,90));

%get boundary coordinates without CL
edge= bwboundaries(imcomplement(imRot));
edge = edge{1};
index = ismember(edge,[xCL' yCL'],'rows');
edge(index,:) = [];

%split bound into two halves
splitPt = mean([max(edge(:,1)) min(edge(:,1))]);
upperBound = edge(edge(:,1)<splitPt,:);
lowerBound = edge(edge(:,1)>splitPt,:);

%rightDCA
minIndex = lowerBound(:,2)<min(lowerBound(:,2))+20;
yyLower = lowerBound(minIndex,1);
[xxLower,sortedIndex] = sort(lowerBound(minIndex,2));
yyLower = yyLower(sortedIndex);
[xxLower,uniqueIndex] = unique(xxLower);
yyLower = yyLower(uniqueIndex);

pLow = polyfit(xxLower,yyLower,3);
polyDer = polyder(pLow);
rightSlope = polyval(polyDer,xxLower(1));
thetaRightDeg = 90+atand(rightSlope);

%leftDCA
minIndex = upperBound(:,2)<min(upperBound(:,2))+20;
yyUpper = upperBound(minIndex,1);
[xxUpper,sortedIndex] = sort(upperBound(minIndex,2));
yyUpper = yyUpper(sortedIndex);
[xxUpper,uniqueIndex] = unique(xxUpper);
yyUpper = yyUpper(uniqueIndex);

pHigh = polyfit(xxUpper,yyUpper,3);
polyDer = polyder(pHigh);
leftSlope = polyval(polyDer,xxUpper(1));
thetaLeftDeg = 90-atand(leftSlope);

end

```

```

function [TimeStar, Spread, EdgeVelocity]=
SpreadCalcs(D_pix, Calibration, FPS, U_o, D_o, t_pastLine)

    TimeArr = 0:length(D_pix)-1;
    Time = TimeArr/FPS; %seconds
    TimeStar=(Time*U_o)/(D_o/1000)+t_pastLine; %Dimensionless Time
    Diameters = D_pix/Calibration;
    Spread = Diameters/D_o;
    EdgeVelocity=zeros(1, length(D_pix));

    %Calc Edge Velocity
    % 4 Point Central Differencing for Interior Points: 0(h4)
    for i=3:length(Diameters)-2

EdgeVelocity(i)=abs(((Diameters(i-2)-8*Diameters(i-1)+8*Diameters(i+1)-
Diameters(i+2)))/(12/FPS)/1000); %2 Pt Cent Diff m/s
        end

        % 3 Point Forward differencing for 1st and 2nd point
        EdgeVelocity(1)=abs((-3*Diameters(1)+4*Diameters(2)-Diameters(3))/(2/
FPS))/1000);
        EdgeVelocity(2)=abs((-3*Diameters(2)+4*Diameters(3)-Diameters(4))/(2/
FPS))/1000);

        % 3 Point Back Differencing for Last and Penultimate points

EdgeVelocity(end)=abs(((Diameters(end-2)-4*Diameters(end-1)+3*Diameters(end))/
(2/FPS))/1000);

EdgeVelocity(end-1)=abs(((Diameters(end-3)-4*Diameters(end-2)+3*Diameters(end-
1)))/(2/FPS))/1000);

    end

```

Published with MATLAB® R2026a

```

function [xLeft, yLeft, xRight,yRight]= EdgeDetection(im,xline,yline)

    the_edge= bwboundaries(imcomplement(im));
    boundary = the_edge{1};
    pointsExclude = NaN(length(boundary),2);
    iter = 1;

    for i = 1:length(xline)
        yVal = yline(i);
        if yVal>1
            for j = 1:1:yVal-1
                pointsExclude(iter,1) = j;
                pointsExclude(iter,2) = xline(i);
                iter = iter+1;
            end
        end
    end

    pointsExclude = rmmissing(pointsExclude);
    ExcludePoints = cat(2,pointsExclude(:,1),pointsExclude(:,2));
    C = setdiff(boundary,ExcludePoints,'rows'); %returns the data in boundary
that is not in yline, with no repetitions. C is in sorted order.

    x=C(:,2); y=C(:,1);
    tempX1 = x(y==1); tempX2 = x(y>1);

    var = ismember(tempX1,tempX2);
    ExcludePoints = cat(2,y(var),x(var));
    C = setdiff(C,ExcludePoints,'rows');
    x=C(:,2);
    y=C(:,1);

    minLeft=min(x);
    maxRight=max(x);

```

Sort Edge into Left and Right

```

%Calc X Midpoint
mid=(maxRight+minLeft)/2;

%Initialize Right Segment
xRight=zeros(1, length(x));
yRight=zeros(1, length(y));

%Initialize Left Segment
xLeft=zeros(1, length(x));
yLeft=zeros(1, length(y));

%Sort Top Edge into Left and Right Segments by x Midpoint
for i = 1:length(x)
    if x(i)<=mid
        xLeft(i)=x(i);

```

```
        yLeft(i)=y(i);
    end
    if x(i)>mid
        xRight(i)=x(i);
        yRight(i)=y(i);
    end
end
```

If position i of xRight is Zero, make cell empty

```
for i = length(xRight):-1:1
    if xRight(i)==0
        xRight(i)=[];
        yRight(i)=[];
    end
end
```

If position i of xLeft is Zero, make cell empty

```
for i = length(xLeft):-1:1
    if xLeft(i)==0
        xLeft(i)=[];
        yLeft(i)=[];
    end
end
```

%Use Sorting Function that sorts Left Edge along path with min distance between pts

```
[idx]=Sorting(xLeft,yLeft);
xLeft=xLeft(idx);
yLeft=yLeft(idx);
```

%Use Sorting Function that sorts Right Edge along path with min distance between pts

```
[idx]=Sorting(xRight,yRight);
xRight=xRight(idx);
yRight=yRight(idx);
```

```
end
```

Published with MATLAB® R2026a

```
function [idx]= Sorting(x,y)
    % Set up loop variables. The first coordinate
    % will be the first coordinate in the new, sorted order.
    yIdx = [1, nan(size(y)-1)];
    xyTemp = [x(:), y(:)];
    xyTemp(1,:) = NaN;
    idx = [1, nan(1, numel(x)-1)];
    counter = 0;
    % Loop through each coordinate and find its nearest neighbor,
    % Then eliminate that coordinate from being chosen again, and
    % then start the loop again finding the nearest neighbor to
    % the point we just located.
    while any(isnan(idx))
        counter = counter+1;
        % find closest coordinate to (x(i),y(i)) that isn't already taken
        [~, idx(counter+1)] = min(pdist2(xyTemp,[x(idx(counter)),
y(idx(counter))]]));
        % Eliminate that from the pool
        xyTemp(idx(counter+1),:) = NaN;
    end
end

end
```

Published with MATLAB® R2026a

```
function im=PreProcessing(I0, ATV)
    %Preprocessing Image
    I = im2uint8( mat2gray(I0)); %Convert Image to 8bit
    BW = imbinarize(I,ATV); %Convert Image to black and white
    BW2 = imcomplement(bwareaopen(imcomplement(BW),1000,4)); %Remove Objects
below 4000 pixels, bwareaopen auto assumes objects are white
    se = strel('disk',50);
    closeBW = imcomplement(imclose(imcomplement(BW2),se));
    BW3 = imcomplement(imfill(imcomplement(closeBW),'holes')); %Fill Holes in
Objects, imfill auto assumes holes are light on black background
    im = BW3; %Change image name to "im"

end
```

Published with MATLAB® R2026a

```
function [m,b] = LinearRegression(x,y)

    n = length(x); Sumx = sum(x); Sumy = sum(y);
    Sumxx = sum(x.*x); Sumxy = sum(x.*y); den = n*Sumxx - Sumx^2;
    m = (n*Sumxy - Sumx*Sumy)/den;
    b = (Sumxx*Sumy - Sumxy*Sumx)/den;

end
```

Published with MATLAB® R2026a

```
function output = convLogArray(input,pts,lowLim,highLim)
    if nargin<3, lowLim=input(1);highLim=input(end);end
    if nargin==1,pts=100;end

    logT = logspace(log10(lowLim),log10(highLim),pts);
    output = NaN(1,length(logT));

    for k = 1:length(output)
        [~,index] = min(abs(logT(k)-input));
        output(k) = index;
    end
    output = unique(output);
end
```

Published with MATLAB® R2026a

```
function [range] = getExcelCol(input)
    if input <= 26
        range = char(64+input);
    else
        input2 = rem(input,26);
        if input2 == 0
            input2 = 26;
        end
        input1 = ceil(input/26)-1;
        range = [char(64+input1),char(64+input2)];
    end
end
```

Published with MATLAB® R2026a

```
function [N,images_folder,image_files] = openFolder(s1,type)
    images_folder = fullfile(s1,type);
    image_files = dir(images_folder);
    image_files(1:2) = [];
    N = length(image_files);
end
```

Published with MATLAB® R2026a